

Why Statistical Learning?

Week 1 — Motivation, Importance, and How It Works (ISLP Ch. 1–2)

Hyun-Hwan Jeong

2026-07-10

Welcome

About this course

- **Instructors:** Ho-min Park & Hyun-Hwan Jeong
- **This session:** led by Hyun-Hwan Jeong
- **Format:** 7 Fridays, 2:00–4:00 PM Central, Jul 10 – Aug 21
- **Textbook:** *An Introduction to Statistical Learning*, Python edition (ISLP)
- Read the chapter **before** class
- Class time = discussion + labs, not lecture
- Every week pairs theory with a runnable notebook

Course syllabus

#	Date	Topics	Chapters
1	Jul 10	Intro + Statistical Learning basics	Ch 1–2
2	Jul 17	Linear Regression; Classification	Ch 3–4
3	Jul 24	Resampling; Regularization	Ch 5–6
4	Jul 31	Moving Beyond Linearity; Tree-Based Methods	Ch 7–8
5	Aug 7	Support Vector Machines; Deep Learning I	Ch 9, Ch 10 (pt 1)
6	Aug 14	Deep Learning II; Unsupervised Learning	Ch 10 (pt 2), Ch 12
7	Aug 21	Survival Analysis; Multiple Testing; wrap-up	Ch 11, Ch 13

Full reading list and slide decks for every week live in the repo README.

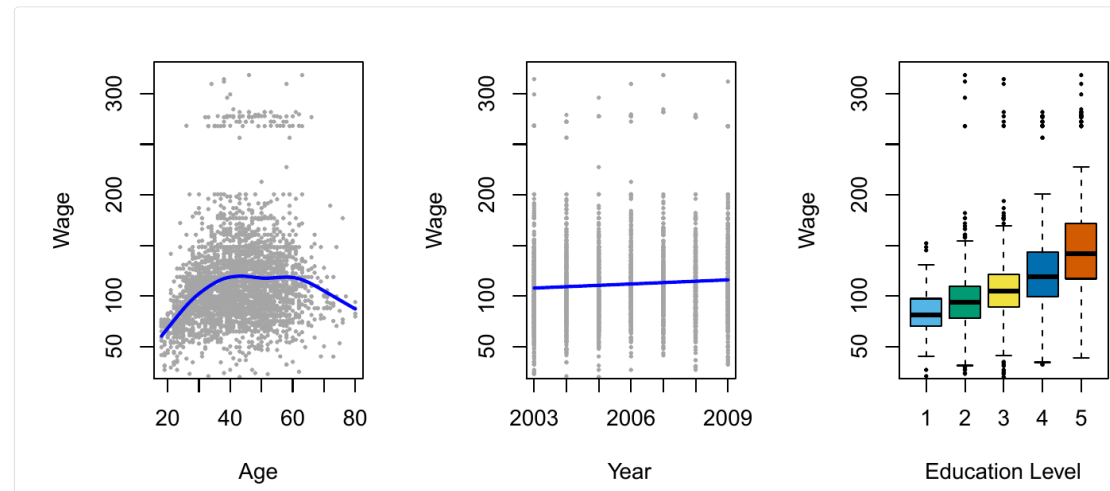
Why Are We Doing This?

Data is everywhere — decisions aren't automatically good

Every field now collects more data than any person can look at by eye. The bottleneck has shifted from *collecting* data to *turning it into decisions*. That's exactly the gap statistical learning fills.

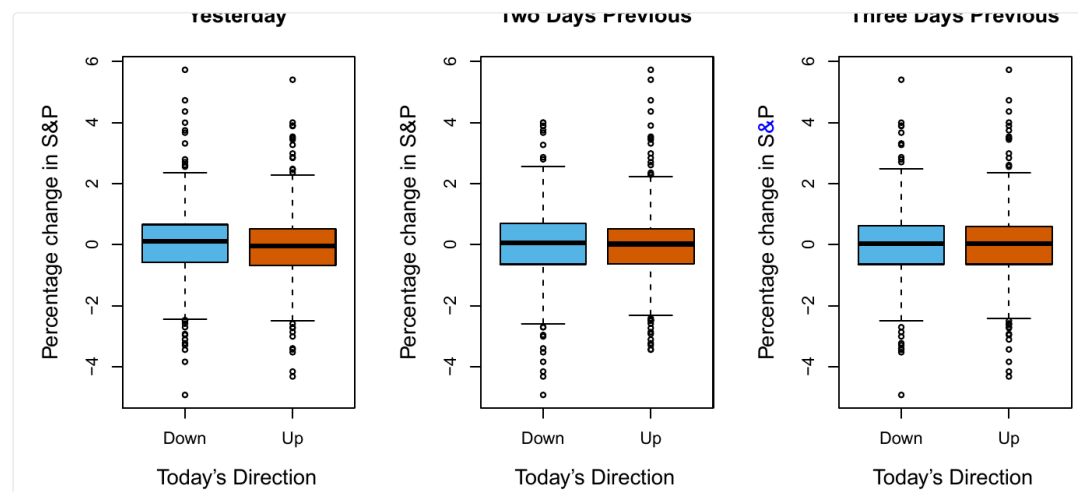
“Statistical learning refers to a vast set of tools for understanding data.” — James, Witten, Hastie, Tibshirani, Taylor (ISLP, Ch. 1)

Wage data — a regression problem



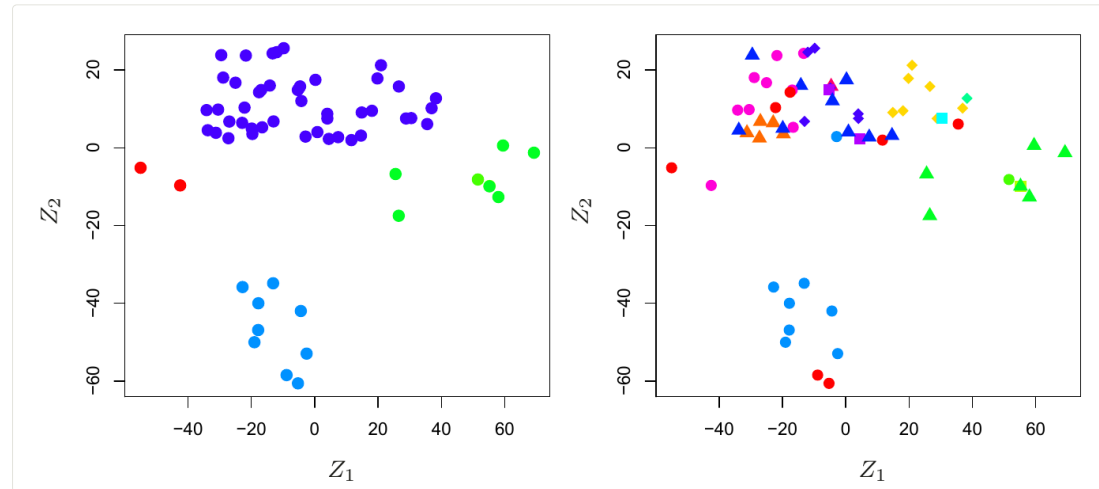
Predict a worker's **wage** — a continuous number — from age, education, and year. We want to estimate it as accurately as possible.

Stock market data — a classification problem



Predict whether the market goes **up or down** tomorrow — a category, not a number — from the past few days' returns.

Gene expression data — no target at all



No output to predict — just **structure** hiding in thousands of measurements. Colors mark the true cancer type; the method never saw those labels, yet the clusters line up with them anyway.

One shape, three questions

Regression, classification, clustering: different questions, but the same underlying machinery — observe data, estimate structure, check how well the estimate generalizes.

That machinery is what this course teaches.

This isn't a fad — it's ~200 years in the making

Early 1800s	Legendre & Gauss formalize least squares — the ancestor of linear regression
1936	Fisher's linear discriminant analysis for classification
1940s–70s	Logistic regression, generalized linear models
1980s	Breiman et al.: classification and regression trees
1990s	Support vector machines ; the term <i>statistical learning</i> emerges
2010s–now	Deep learning, huge data, cheap compute — old ideas at new scale

Almost everything in this course is a 20th-century statistical idea, made newly practical by 21st-century data and compute.

Why you, specifically

- These methods show up **inside** the tools you already use: recommendation systems, medical risk scores, fraud detection, A/B tests, genomics pipelines.
- Being able to read a model's assumptions (and know when they break) is what separates *using* a black box from *trusting* one.
- The skills transfer directly to the Python data-science stack ([scikit-learn](#), [statsmodels](#)) you'll use outside this course.

Why It Is Important

It's the common language of modern data work

- **Science** — genomics, astrophysics, climate models
- **Medicine** — diagnosis support, survival/risk prediction
- **Business** — demand forecasting, pricing, churn
- **Technology** — search ranking, spam filtering, the models behind LLMs



One toolkit, many domains.

The same bias-variance trade-off that governs a linear regression also governs a neural network. Learn the trade-off once.

Getting it wrong is expensive

- An **overfit** model looks great on the data you trained it on and fails the moment it meets new data — silently, until it's in production.
- An **overly rigid** model misses real signal and gives you false confidence in a simple story.
- Statistical learning gives you the *vocabulary and diagnostics* (train/test split, cross-validation, bias-variance) to tell which mistake you're making — and fix it.

How It Works

First, the vocabulary

The book's running example: the **Advertising** data — *sales* of a product in $n = 200$ markets, alongside *TV*, *radio*, and *newspaper* ad budgets.

- What we predict **from** — the budgets — are the **inputs**: also called **predictors**, *features*, or *independent variables* (X_1, X_2, X_3)
- What we predict — *sales* — is the **output**: the **response** or *dependent variable* (Y)
- **Notation for the whole course**: n = number of observations, p = number of predictors, x_{ij} = predictor j for observation i

The book (and every paper you'll ever read) uses these names interchangeably — hear “feature,” think “predictor,” think X .

The basic setup

We assume there's some relationship between an input X and an output Y :

$$Y = f(X) + \epsilon$$

- f — the **true, unknown** systematic relationship we want to learn
- ϵ — random noise, independent of X , mean zero
- **Statistical learning = a set of approaches for estimating f**

Everything this course covers — linear regression, trees, neural networks — is a different strategy for estimating the same f .

Two reasons to estimate f

Prediction

We only care about the output. f can be a “black box” as long as $\hat{Y} = \hat{f}(X)$ is accurate.

e.g. will this loan default?

Inference

We want to understand **how** Y changes with X : which predictors matter, and how. f must stay interpretable.

e.g. which drug dosage actually helps?

Most real projects are a mix of both — and the mix determines which method you should reach for.

Two kinds of error — only one is your fault

Even a *perfect* estimate of f would not predict perfectly:

$$E(Y - \hat{Y})^2 = \underbrace{\left[f(X) - \hat{f}(X) \right]^2}_{\text{reducible}} + \underbrace{\text{Var}(\epsilon)}_{\text{irreducible}}$$

- **Reducible error** shrinks as $\hat{f}$ improves — every method in this course is a strategy for reducing it.
- **Irreducible error** comes from unmeasured variables and inherent randomness. **No model, however clever, gets below it.**
- In practice that floor is *unknown* — a model that plateaus leaves you guessing whether you've hit the floor or just a bad $\hat{f}$.

How do we estimate f ?

Parametric methods

1. Assume a functional **form** (e.g. $f(X) = \beta_0 + \beta_1 X$)
2. Estimate a small set of **parameters** from the training data

Simpler, needs less data, but wrong if the assumed form is wrong.

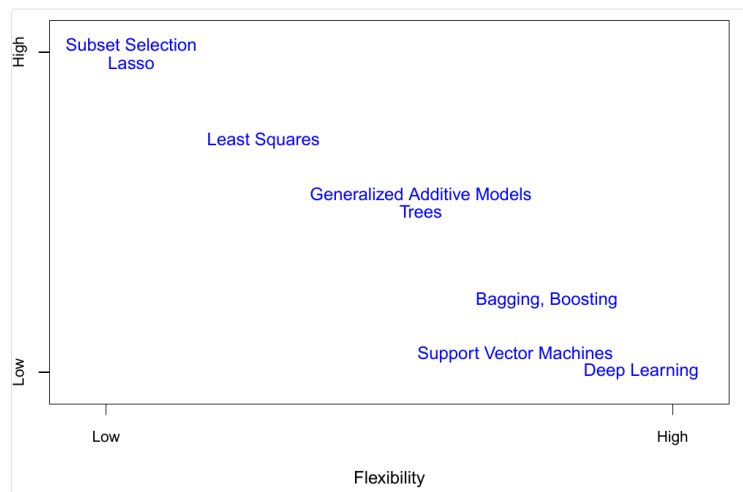
Every method in this course sits somewhere on this spectrum — and where it sits determines how much data it needs and how easy it is to explain.

Non-parametric methods

No explicit functional form — let the data determine the shape of f directly (e.g. splines, trees, KNN).

More flexible, needs more data, harder to interpret.

The flexibility spectrum, mapped



Every method in this course has a home on this plot. Moving right buys flexibility at the cost of interpretability — there's no free lunch.

Two kinds of learning problems

Supervised

Every observation has a **labeled outcome** Y . We fit f to match inputs to known answers.

Regression (numeric Y) · Classification (categorical Y)

Unsupervised

No labeled outcome — only inputs X . We look for structure: groups, patterns, lower-dimensional summaries.

Clustering · Dimension reduction

Regression vs. classification

- Y **quantitative** (a number: wage, price, temperature) → a **regression** problem
- Y **qualitative** (a category: default/no-default, up/down) → a **classification** problem

The distinction matters because we measure “getting it right” differently — a wrong number is off by *some amount*; a wrong category is just *wrong*. That difference drives which methods and which accuracy metrics apply.

Measuring the quality of fit

For regression, the standard yardstick is **mean squared error**:

$$\text{MSE} = \frac{1}{n} \sum_{i=1}^n \left(y_i - \hat{f}(x_i) \right)^2$$

- Small on the **training data** is easy — and not the point.
- What we actually care about is small **test MSE**: how well $\hat{f}$ predicts *new*, unseen observations.
- A model can look excellent on training data and terrible on test data — that gap is overfitting, made precise.

The bias-variance trade-off, precisely

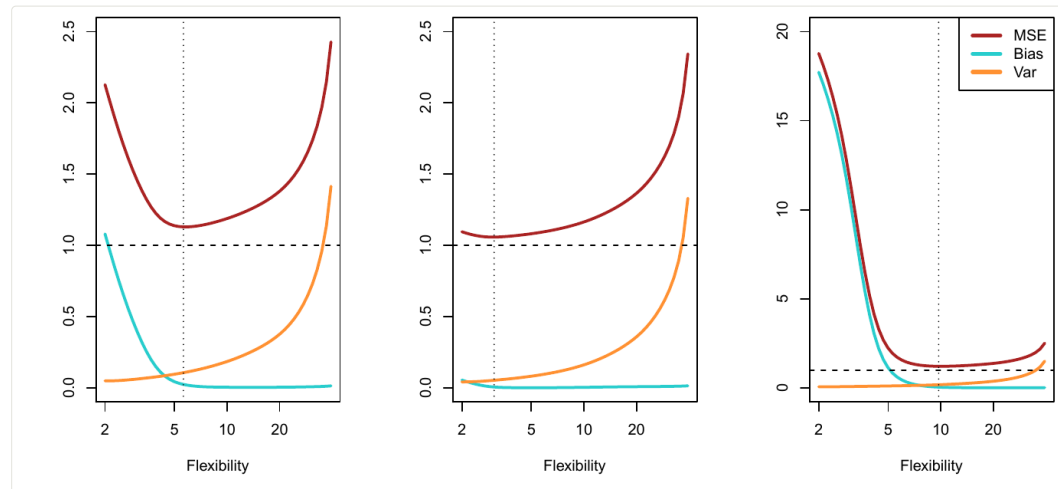
Expected test error at a point x_0 decomposes into three pieces:

$$E \left[\left(y_0 - \hat{f}(x_0) \right)^2 \right] = \text{Var}(\hat{f}(x_0)) + \left[\text{Bias}(\hat{f}(x_0)) \right]^2 + \text{Var}(\epsilon)$$

- **Variance** — how much $\hat{f}$ would change on a different training set. Flexible models (trees, splines, deep nets) vary a lot.
- **Bias** — error from approximating a complex reality with a simpler model. Simple models (linear regression, LDA) tend to be biased.
- $\text{Var}(\epsilon)$ — irreducible noise. No model beats it.

Minimizing test error means *trading* variance against bias — not eliminating either one.

What it looks like in practice



Squared bias (cyan) falls and variance (orange) rises as flexibility increases. Test MSE (red) is their sum — it bottoms out in the middle, not at either extreme.

The classification setting

Instead of MSE, classification uses the **error rate**:

$$\frac{1}{n} \sum_{i=1}^n I(y_i \neq \hat{y}_i)$$

The theoretical best possible classifier — the **Bayes classifier** — assigns each x_0 to the class j that maximizes $\Pr(Y = j \mid X = x_0)$. It's a gold standard we can't compute directly (we don't know the true conditional distribution), but we can approximate it.

The floor, again: the Bayes error rate

Even the Bayes classifier isn't perfect — *best possible* still makes errors wherever the classes genuinely overlap:

$$\text{Bayes error rate} = 1 - E \left(\max_j \Pr(Y = j \mid X) \right)$$

- That error is built into the **problem**, not the method — it's the classification twin of the **irreducible error** from the regression setting.
- Every real classifier (KNN, next) is judged by how close it gets to this floor — never by whether it reaches zero.

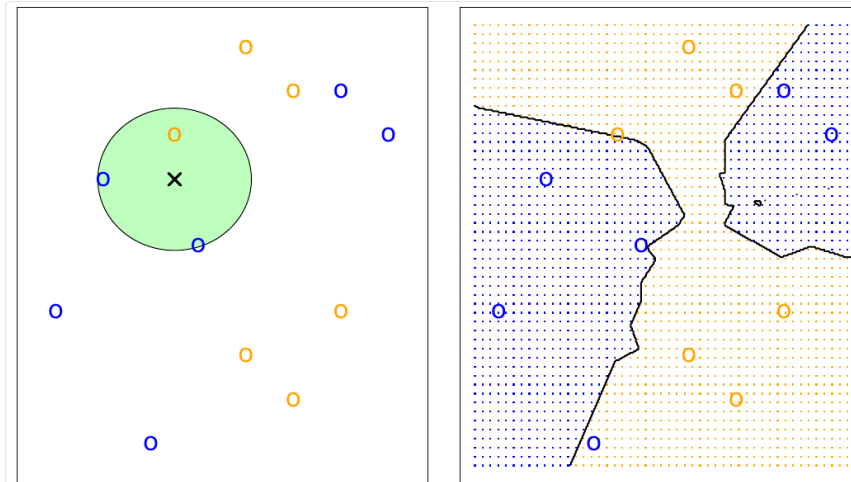
K-Nearest Neighbors

KNN approximates the Bayes classifier non-parametrically: look at the K closest training points to x_0 and vote.

$$\Pr(Y = j \mid X = x_0) \approx \frac{1}{K} \sum_{i \in \mathcal{N}_0} I(y_i = j)$$

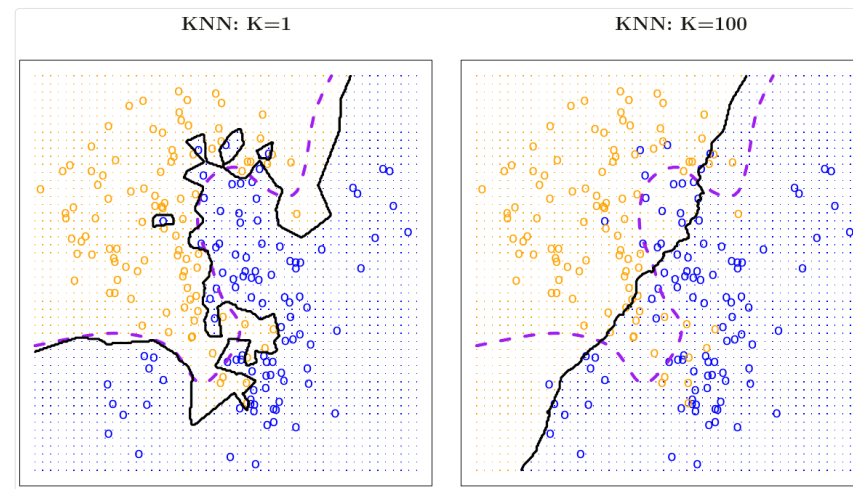
Small $K \rightarrow$ flexible, low bias, high variance. Large $K \rightarrow$ the opposite. Same trade-off, new clothes.

KNN in action ($K = 3$)



Left: the 3 nearest neighbors of the test point (X) vote — 2 blue beats 1 orange. Right: repeating this at every point traces out the decision boundary.

Choosing K is choosing flexibility



$K = 1$ (left) chases every training point — low bias, high variance. $K = 100$ (right) barely bends — high bias, low variance. The purple dashed curve is the true (Bayes) boundary neither one quite matches.

Where the next 7 weeks go

Week	Focus
1	Foundations — what we just covered
2	Regression & classification: fitting f to data
3	Resampling & regularization: trusting your estimate of f
4	Beyond straight lines: splines, trees
5–6	Support vector machines, deep learning
7	Survival analysis, unsupervised learning, multiple testing

Every week's lab makes the ideas from that week's slides concrete in Python.

Getting Started

Before next Friday

1. Read **ISLP Ch. 3–4** ([week2/ch03-04-regression-classification.pdf](#)) — Linear Regression & Classification, for Jul 17
2. If you haven't already, set up a Python environment with [jupyter](#), [numpy](#), [pandas](#), [scikit-learn](#), [statsmodels](#), and the [ISLP](#) package
3. If you didn't finish running today's Ch. 2 lab — *Introduction to Python* — in class, catch up on it before next week's lab builds on it
4. Bring questions; we'll spend class time on discussion + the lab, not re-reading slides

Questions?

See you next Friday.